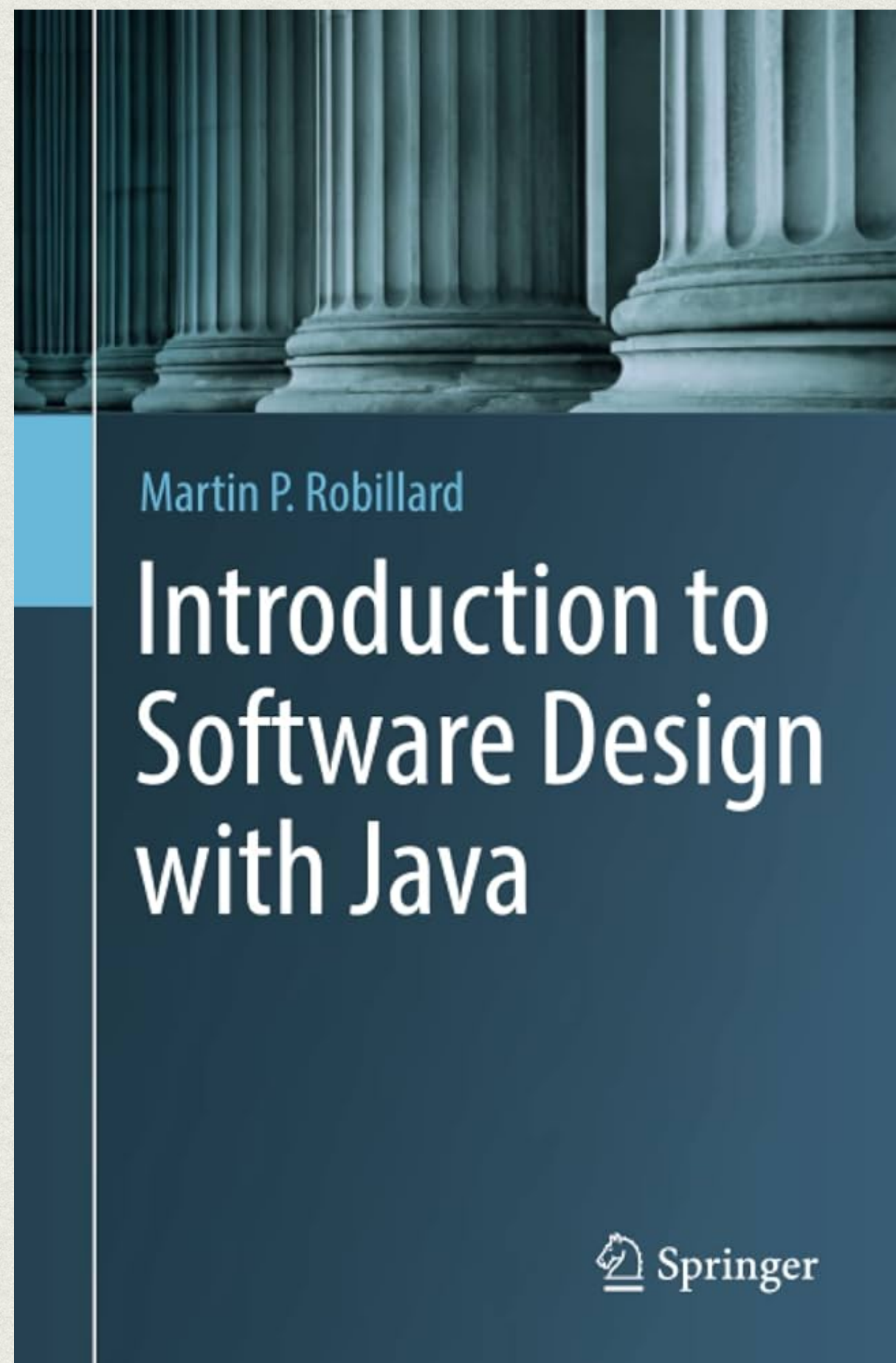




M1 MIAGE / MAPC INTRODUCTION

Pascal POIZAT

RÉFÉRENCES



Conception Logicielle
Site web du cours INF5153 de l'UQAM

- [Retourner à la page d'accueil](#)

Introduction

Notes de cours

- [Introduction au cours](#)
- [Écosystème de développement](#)
- [Schotten Totten](#)

Capsules Vidéos

Introduction au cours

Introduction du cours de Conception Logicielle [IN...]

Partager

Génie logiciel : Conception

Regarder sur YouTube

Sébastien Mosser - INF5153
Chapitre 0 - Capsule 1
Automne 2020

ace

créé par photos: Pixabay

LES 12 CADEAUX DE NOËL

- On the first day of Christmas my true love sent to me
a partridge in a pear tree

On the second day of Christmas my true love sent to me
two turtle doves,
and a partridge in a pear tree.

On the third day of Christmas my true love sent to me
three French hens,
two turtle doves,
and a partridge in a pear tree.

...

LES 12 CADEAUX DE NOËL

- On the first day of Christmas my true love sent to me
a partridge in a pear tree

On the second day of Christmas my true love sent to me
two turtle doves,
and a partridge in a pear tree.

On the third day of Christmas my true love sent to me
three French hens,
two turtle doves,
and a partridge in a pear tree.

...



```
1 #include <stdio.h>
2 main(t,_,a)char *a;{return!0<t?t<3?main(-79,-13,a+main(-87,1-_,
3 main(-86,0,a+1)+a)):1,t<_?main(t+1,_,a):3,main(-94,-27+t,a)&&t==2?_<13?
4 main(2,_+1,"%s %d %d\n"):9:16:t<0?t<-72?main(_,t,
5 "@n'+,#'/*{}w+/w#cdnr/+,{}r/*de}+,/*{*+,/w{%,/w#q#n+,/#{l,+,/n{n+,/+#n+,/#\
6 ;#q#n+,/+k#;*+,/'r :'d*'3,}{w+K w'K:'+'e#';dq#'l \
7 q#'+d'K#!/+k#;q#'r}eKK#}w'r}eKK{nL}'/#;#q#n')}{nL}'/+#n';d}rw' i;# \
8 ){nL}!/n{n#'; r{#w'r nc{nL}'/{l,+'K {rw' iK{;[{nL}]/w#q#n'wk nw' \
9 iwk{KK{nL}!/w{% 'l##w# ' i; :{nL}'/{q# 'ld;r'}{nlwb!/*de}'c \
10 ;;{nL}'-{}rw}]/+,}##'*}#nc,',#nw}]/+kd'+e}+;#'rdq#w! nr'/ ' ) }+}{rl#'{n' ' )# \
11 }'+}##(!!/" )
12 :t<-50?_==*a?putchar(31[a]):main(-65,_,a+1):main((*a=='/')+t,_,a+1)
13 :0<t?main(2,2,"%s"):a=='/'||main(0,main(-61,*a,
14 "!ek;dc i@bK'(q)-[w]*%n+r3#l,{ }:\nuwloca-0;m .vpbks,fxntdCeghiry"),a+1);}
```

code par I. Philipps, 1988

LES 12 CADEAUX DE NOËL

- On the first day of Christmas my true love sent to me
a partridge in a pear tree

On the second day of Christmas my true love sent to me
two turtle doves,
and a partridge in a pear tree.

On the third day of Christmas my true love sent to me
three French hens,
two turtle doves,
and a partridge in a pear tree.

...

un problème ?

```
1 #include <stdio.h>
2 main(t,_,a)char *a;{return!0<t?t<3?main(-79,-13,a+main(-87,1-_,
3 main(-86,0,a+1)+a)):1,t<_?main(t+1,_,a):3,main(-94,-27+t,a)&&t==2?_<13?
4 main(2,_,+1,"%s %d %d\n"):9:16:t<0?t<-72?main(_,t,
5 "@n'+,#'/*{}w+/w#cdnr/+,{}r/*de}+,/*{*+,/w{%,/w#q#n+,/#{l,+,/n{n+,/+#n+,/#\
6 ;#q#n+,/+k#;*,/'r : 'd*'3,}{w+K w'K:'+'e#';dq#'l \
7 q#'+d'K#!/+k#;q#'r}eKK#}w'r}eKK{nL}'/#;#q#n')}{nL}'/+#n';d}rw' i;# \
8 ){nL}!/n{n#'; r{#w'r nc{nL}'/{l,+'K {rw' iK{;[{nL}'/w#q#n'wk nw' \
9 iwk{KK{nL}!/w{% 'l##w# ' i; :{nL}'/*{q#'ld;r'}{nlwb!/*de}'c \
10 ;;{nL}'-{}rw]'/+,}##'*}#nc,',#nw]'/+kd'+e}+;#'rdq#w! nr'/ ' ) }+}{rl#'{n' ' )# \
11 }'+}##(!!/" )
12 :t<-50?_==*a?putchar(31[a]):main(-65,_,a+1):main((*a=='/')+t,_,a+1)
13 :0<t?main(2,2,"%s"):a=='/'||main(0,main(-61,*a,
14 "!ek;dc i@bK'(q)-[w]*%n+r3#l,{ }:\nuwloca-0;m .vpbks,fxntdCeghiry"),a+1);}
```

code par I. Philipps, 1988

LES 12 CADEAUX DE NOËL

- On the first day of Christmas my true love sent to me
a partridge in a pear tree

On the second day of Christmas my true love sent to me
two turtle doves,
and a partridge in a pear tree.

On the third day of Christmas my true love sent to me
three French hens,
two turtle doves,
and a partridge in a pear tree.

...



```
1 static final int MAX_DAYS = 3;
2 static final String[] DAYS = { "first", "second", "third" };
3 static IntFunction<String> firstLine = d → String
4     .format("On the %s day of Christmas, my true love sent to me:%n", DAYS[d]);
5
6 static final String[] GIFTS = { "a partridge in a pear tree", "two turtle doves", "three French hens" };
7 static IntFunction<String> allGifts;
8 static {
9     allGifts = d → switch (d) {
10         case 0 → "and " + GIFTS[0];
11         default → GIFTS[d] + "\n" + allGifts.apply(d - 1);
12     };
13 }
14
15 static final String SEPARATOR = "\n\n";
16
17 static String poem() {
18     StringBuilder poem = new StringBuilder(firstLine.apply(0) + GIFTS[0] + SEPARATOR);
19     for (int day = 1; day < MAX_DAYS; day++) {
20         poem.append(firstLine.apply(day) + allGifts.apply(day) + SEPARATOR);
21     }
22     return poem.toString();
23 }
```


LES 12 CADEAUX DE NOËL

- On the first day of Christmas my true love sent to me
a partridge in a pear tree

On the second day of Christmas my true love sent to me
two turtle doves,
and a partridge in a pear tree.

On the third day of Christmas my true love sent to me
three French hens,
two turtle doves,
and a partridge in a pear tree.

...

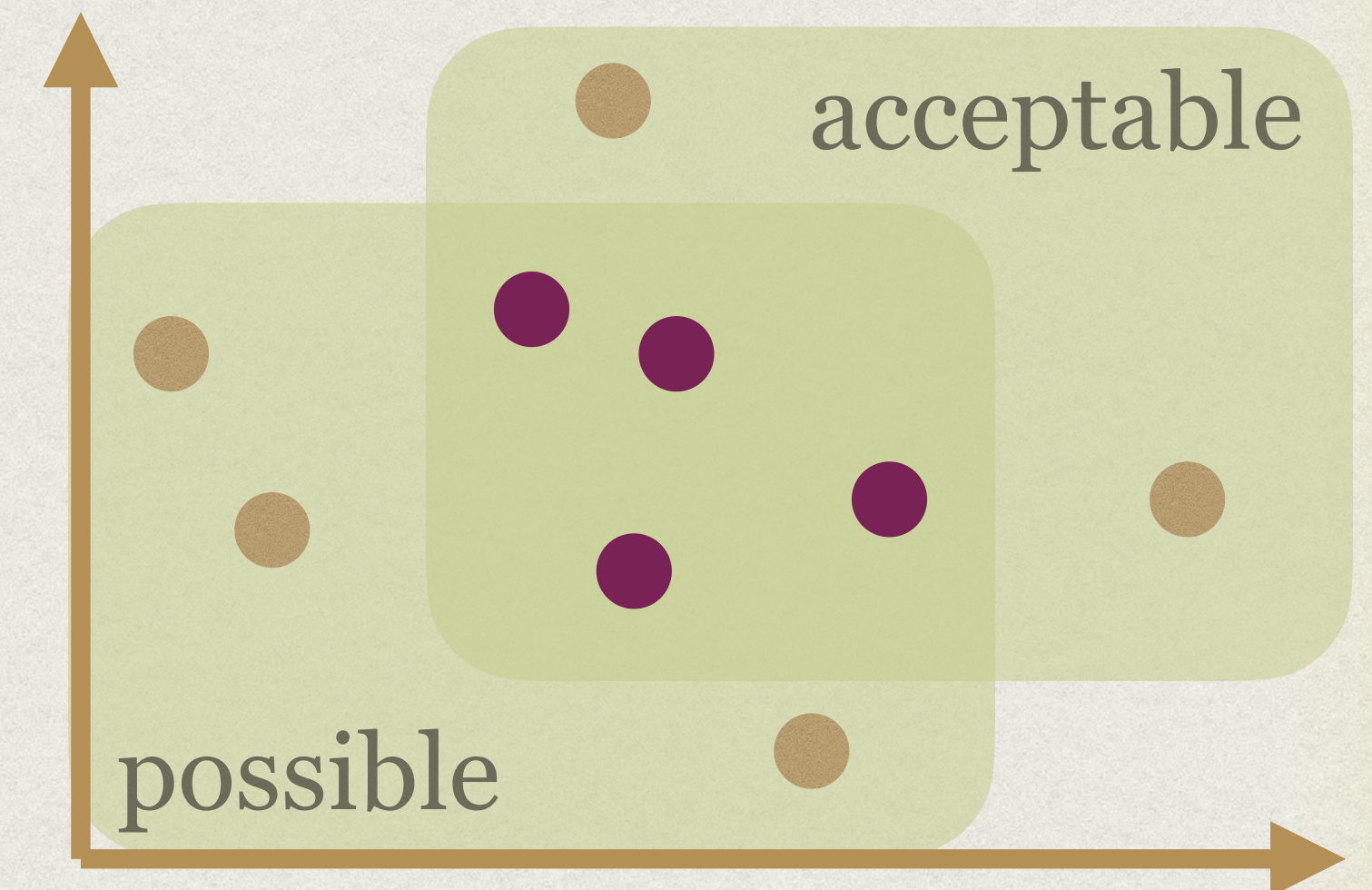
```
1 public class DayInfo {
2     private DayInfo() {}
3
4     public static final int MAX_DAYS = 3;
5     private static final String[] DAYS = { "first", "second", "third" };
6     private static final String[] GIFTS = { "a partridge in a pear tree", "two turtle doves", "three French hens" };
7
8     public static String day(int day) { return (day ≥ 0 && day < MAX_DAYS) ? DAYS[day] : null; }
9     public static String gift(int day) { return (day ≥ 0 && day < MAX_DAYS) ? GIFTS[day] : null; }
10 }
11
12 static IntFunction<String> firstLine = d → String
13     .format("On the %s day of Christmas, my true love sent to me:%n", DayInfo.day(d));
14
15 static IntFunction<String> allGifts;
16 static {
17     allGifts = d → switch (d) {
18         case 0 → "and " + DayInfo.gift(d);
19         default → DayInfo.gift(d) + "\n" + allGifts.apply(d - 1);
20     };
21 }
22
23 static String poem() {
24     final String SEPARATOR = "\n\n";
25     StringBuilder poem = new StringBuilder(firstLine.apply(0) + DayInfo.gift(0) + SEPARATOR);
26     for (int day = 1; day < DayInfo.MAX_DAYS; day++) {
27         poem.append(firstLine.apply(day) + allGifts.apply(day) + SEPARATOR);
28     }
29     return poem.toString();
30 }
31
```


CONCEPTION (*DESIGN*)

- nombreuses définitions (dépend du focus)
on suit ici [**Robillard, 2017**].
- construction d'**abstractions** (données et calculs)
et **organisation** de ces abstractions
en une application logicielle fonctionnelle

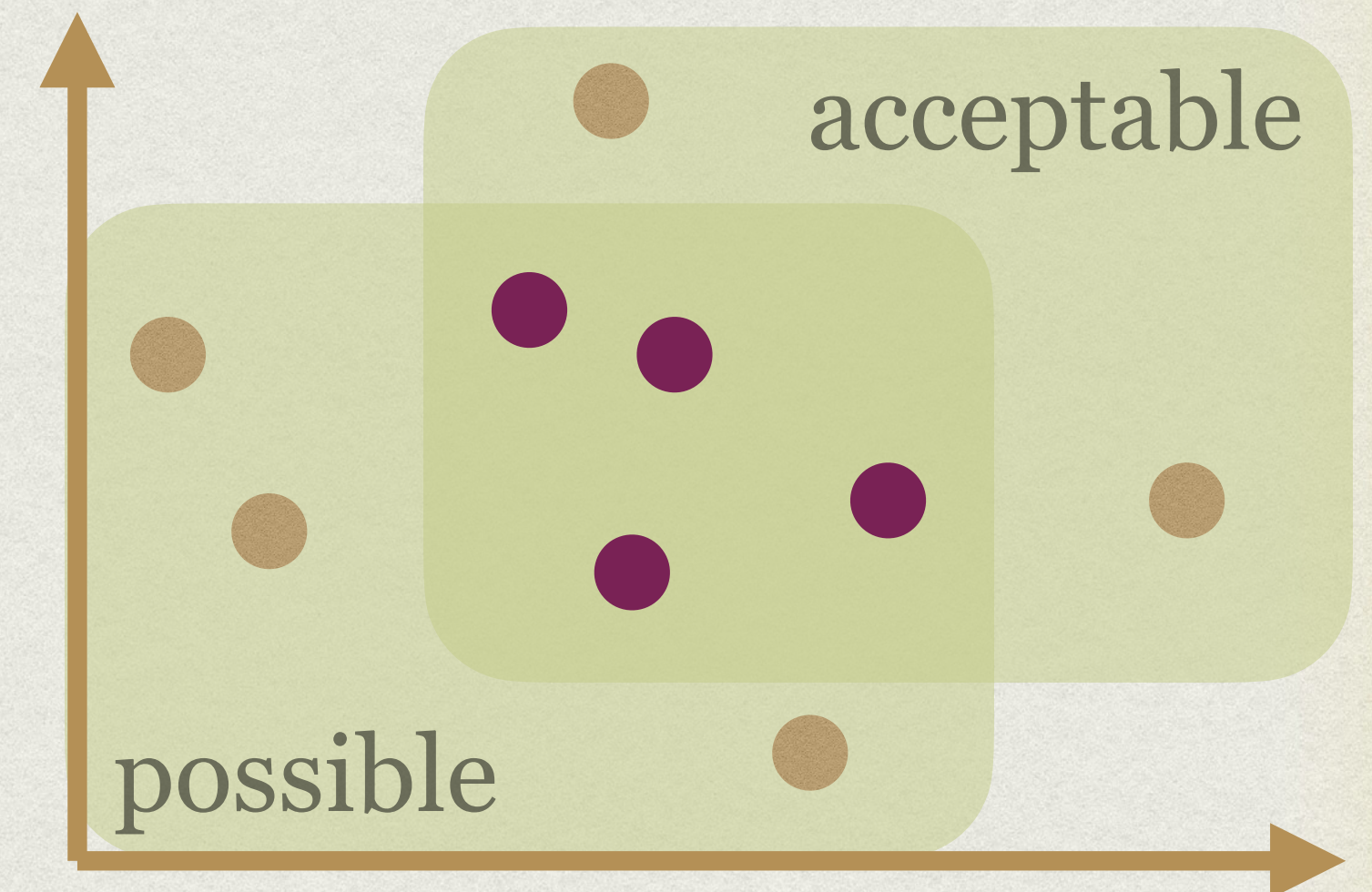
PROCESSUS DE CONCEPTION

- le **processus de conception** est un processus de **prise de décisions** conduisant à l'obtention d'une **solution** (modèle plus ou moins abstrait)
- **espace de conception** à n dimensions (choix)
- **compromis** de conception
- existence de solutions **possibles** (ou non) et **acceptables** (ou non)



DIFFICULTÉ

- **incertitude** (critères de décision non encore connus, en grand nombre)
- caractère parfois **approximatif** de l'évaluation d'un critère de qualité d'une solution
- processus **heuristique** et **itératif** guidé par l'expérience
- poids des critères de qualité dépendant du **contexte** de conception par exemple : robustesse vs. durabilité d'une solution



CONCEPTION ET PROCESSUS DE DÉVELOPPEMENT

- la conception est seulement l'**une** des activités au sein des processus de développement logiciel
- impact de **bonnes pratiques** pour de bonnes solutions
- **approche évolutive**
itérations, refactoring, gestion de la dette technique
- **conventions** et **principes**
choix des noms, casse, indentation, annotations, commentaires
DRY, KISS, SOLID, STUPID

PRINCIPES : DRY

- « **Don't Repeat Yourself** » (Hunt & Thomas, 1999)
- **limiter la répétition**
1 élément de connaissance = 1 représentation unique
- lien avec bogues (dupliquées) et maintenance (plus difficile)
- préférer passer par des (dépendre d') **abstractions** moins changeantes
mais attention à l'abstraction / optimisation prématurée

PRINCIPES : KISS

- « **Keep It Simple, Stupid!** » (origine ?)
- la **simplicité** comme l'un des objectifs de la conception
- éviter la complexité, à nouveau impact sur la maintenance

PRINCIPES : SOLID

- ensemble de principes (par B. Liskov, R. C. Martin, B. Meyer)
- **Single responsibility principle (SCP)**
responsabilité unique, une unique raison de changer
- **Open/closed principle (OCP)**
ouverture à l'extension, fermeture à la modification, permettre extension sans modification
- **Liskov substitution principle (LSP)**
substituabilité, notion sémantique, $S < T$ implique que ce qui est vrai pour tout T doit l'être pour tout S
- **Interface segregation principle (ISP)**
ne pas dépendre de méthodes que l'on n'utilise pas
- **Dependency inversion principle (DIP)**
les abstractions ne dépendent pas des détails - les détails dépendent des abstractions

PRINCIPES : STUPID

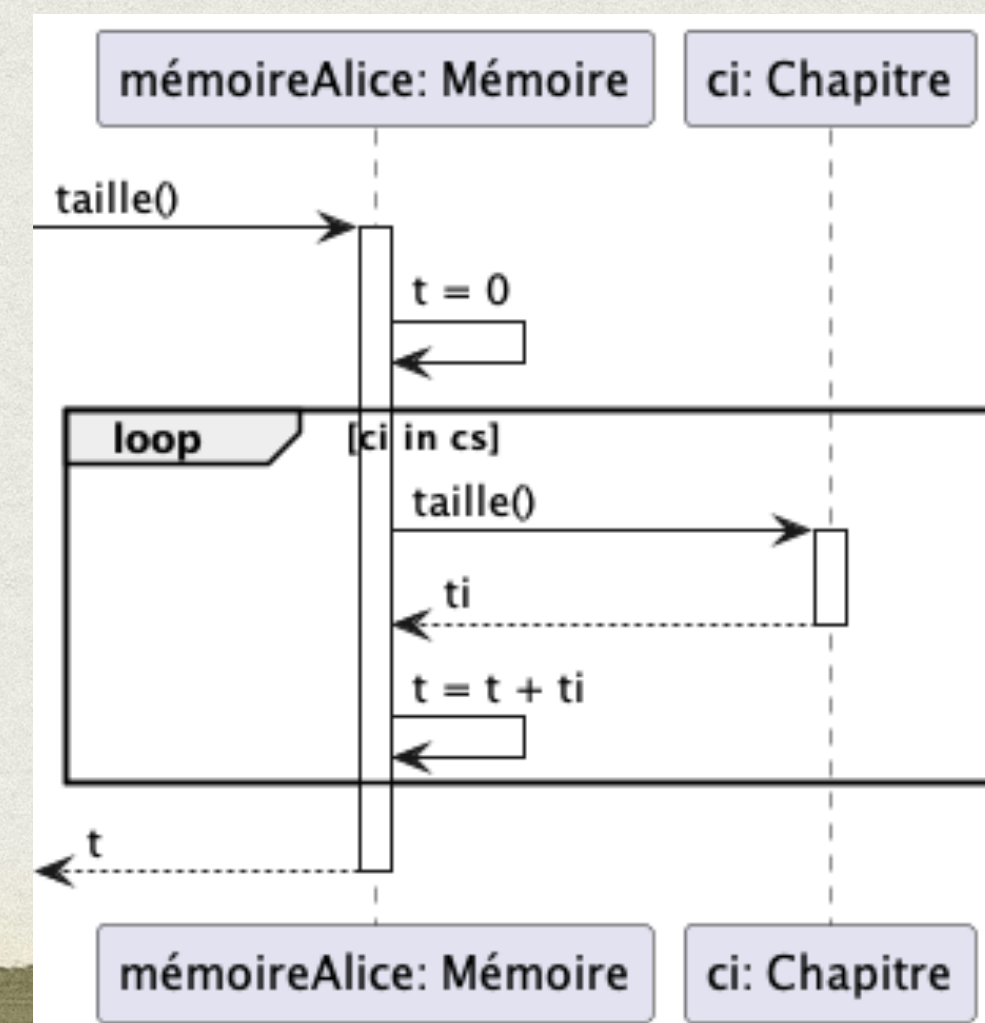
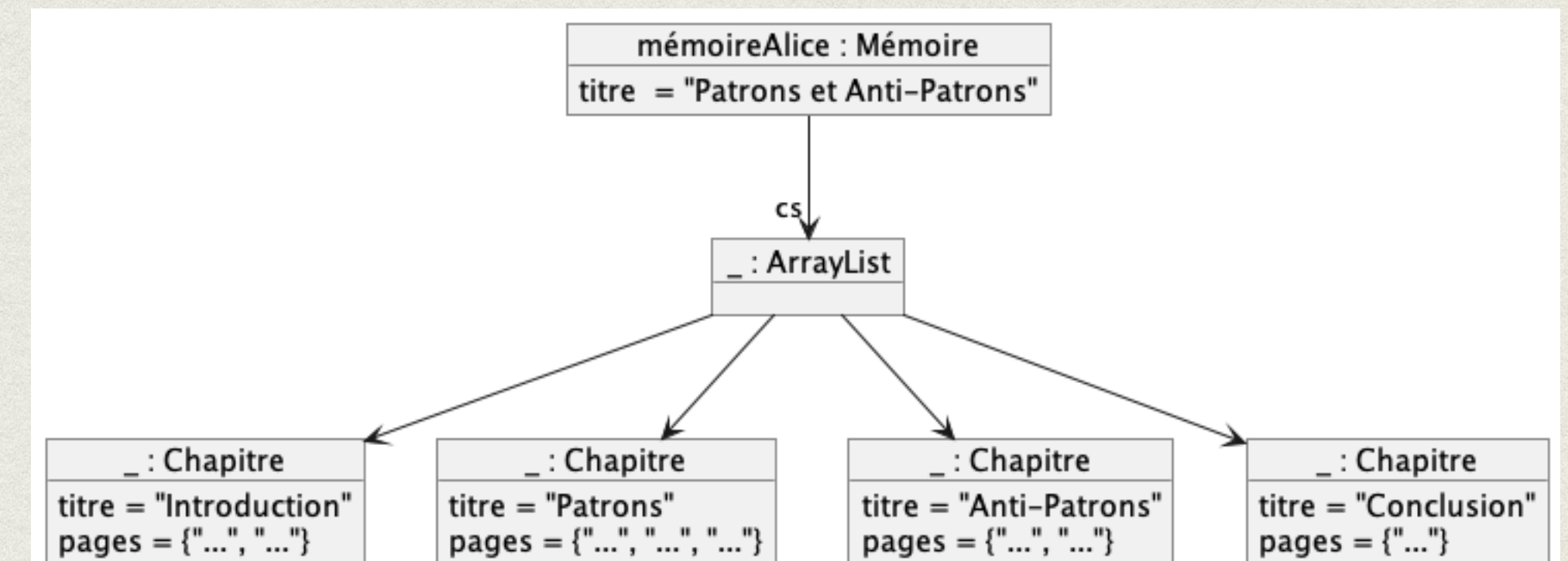
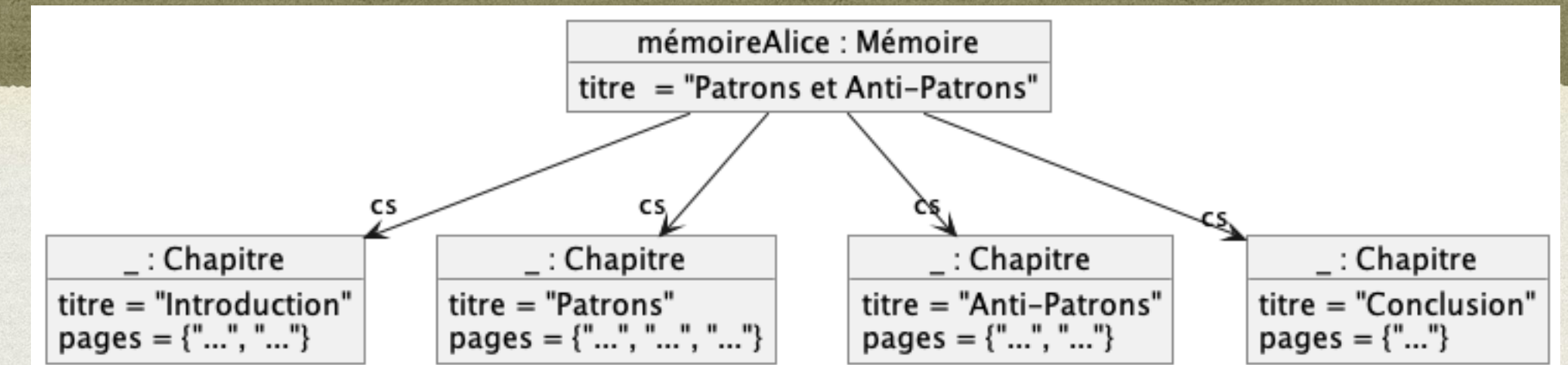
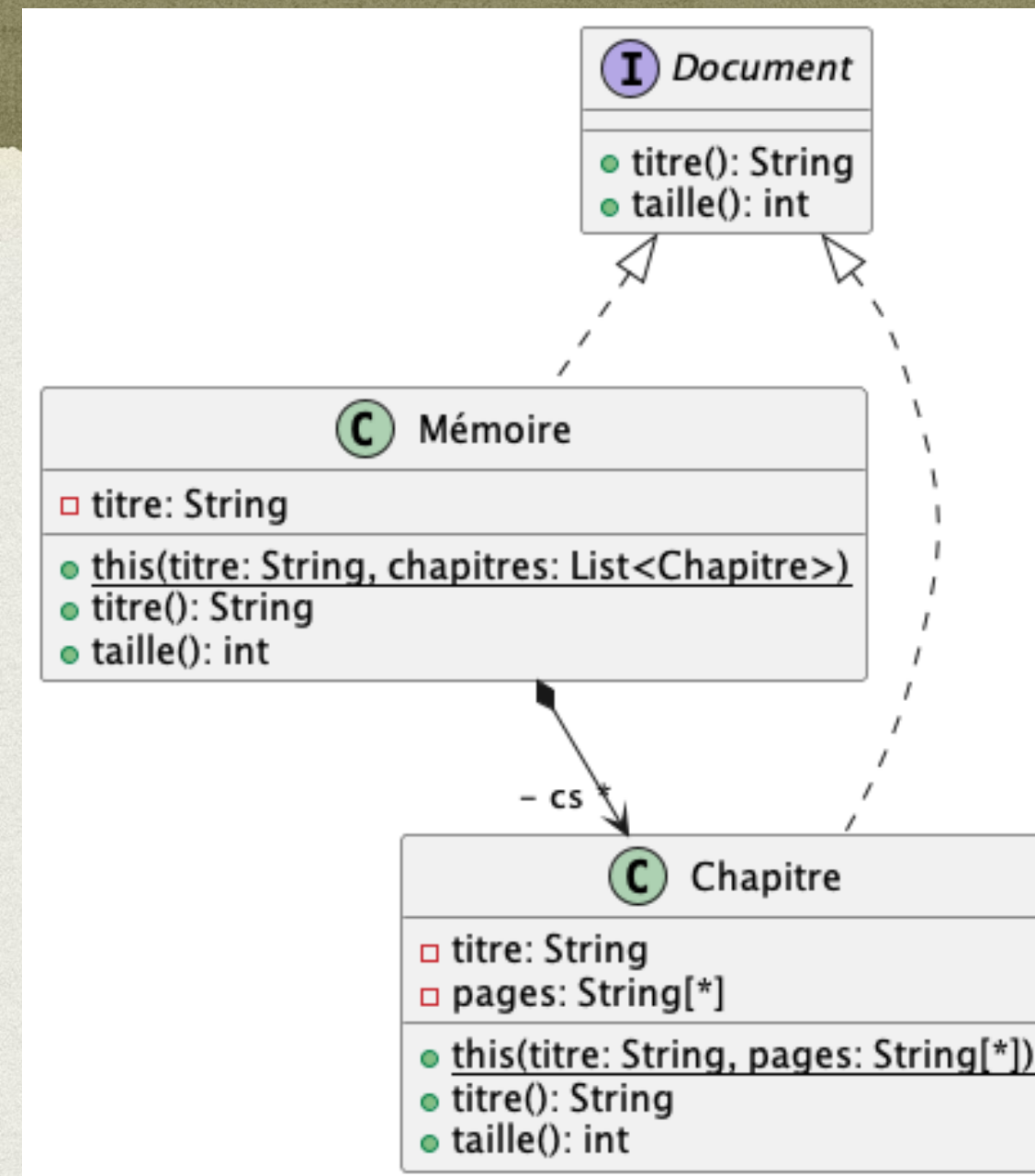
- ensemble de pratiques considérées comme mauvaises (W. Durand, 2013)
<https://williamdurand.fr/2013/07/30/from-stupid-to-solid-code/>
liées à la notion d'état global, de fort couplage, de perte de testabilité
- **Singleton**
- **Tight (strong) coupling**
- **Untestability**
- **Premature optimization**, « root of all evil » (Knuth, 1974)
- **Indescribable naming**
- **Duplication**

CAPTURE DES CHOIX DE CONCEPTION

- **outils dédiés**
plateformes de discussion, systèmes de gestion de versions
- **code source**
choix encodés et potentiellement vérifiables par le compilateur
mais pas la logique ayant conduit aux choix
- **documents de conception** de différentes natures (GPML, DSML)
y compris utilisation de modèles génératifs

MODÉLISATION : UML

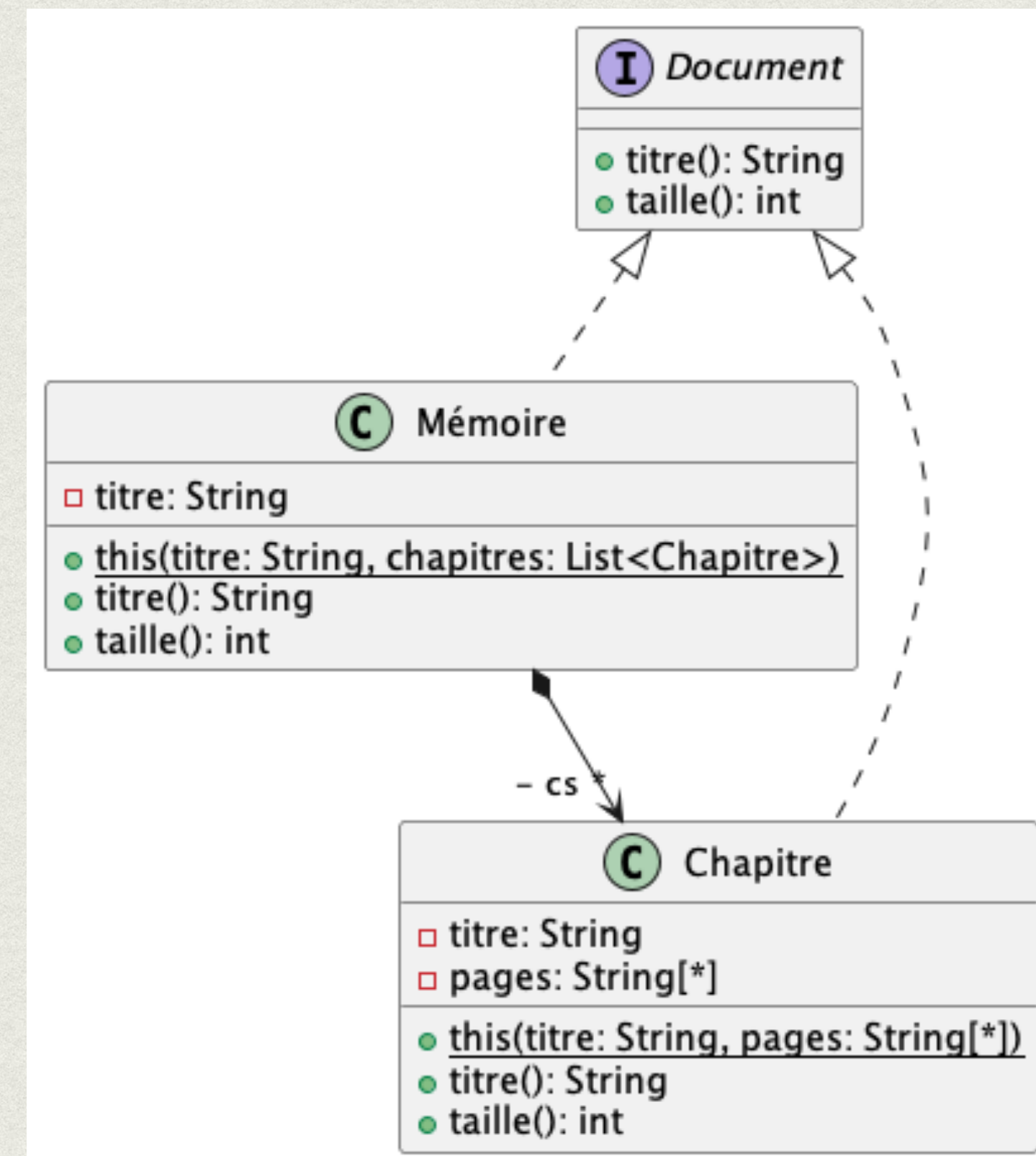
- choix de l'**outil**
bibliothèque texte vers modèle
outil plus « lourd »
- choix des **diagrammes**
diagrammes d'objets
diagrammes de classe
diagrammes d'états ou de séquence
- choix du **niveau de détail**
on cible une partie du système
on donne le détail nécessaire
UML vs Java



MODÉLISATION : UML



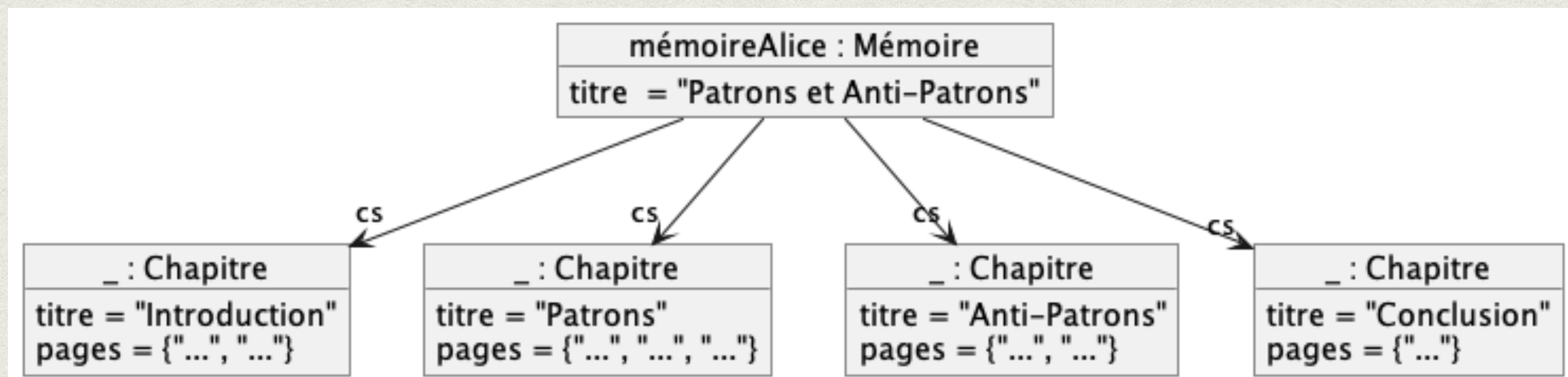
```
1  @!include ../commons.style
2  @startuml dc_example
3  interface Document {
4      + titre(): String
5      + taille(): int
6  }
7  class Mémoire implements Document {
8      - titre: String
9      + {static} this(titre: String, chapitres: List<Chapitre>)
10     + titre(): String
11     + taille(): int
12 }
13 class Chapitre implements Document {
14     - titre: String
15     - pages: String[*]
16     + {static} this(titre: String, pages: String[*])
17     + titre(): String
18     + taille(): int
19 }
20 Mémoire *--> "- cs *" Chapitre
21 @enduml
```



MODÉLISATION : UML



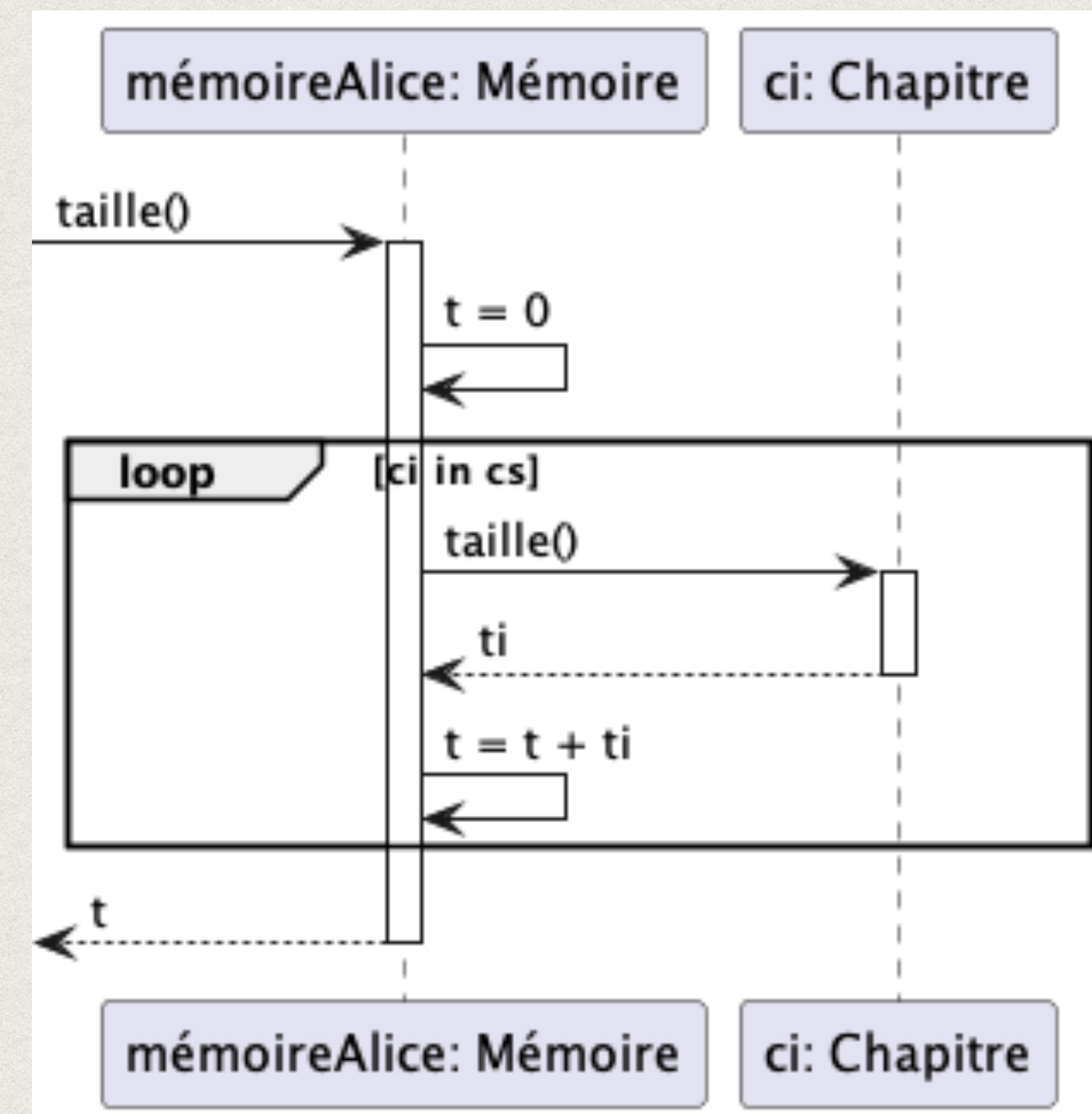
```
1  @!include ../commons.style
2  @startuml do_example2
3
4  object "mémoireAlice : Mémoire" as mA
5  mA : titre = "Patrons et Anti-Patrons"
6
7  object "_ : Chapitre" as c1
8  c1 : titre = "Introduction"
9  c1 : pages = {"...", "..."}
10
11 object "_ : Chapitre" as c2
12 c2 : titre = "Patrons"
13 c2 : pages = {"...", "...", "..."}
14
15 object "_ : Chapitre" as c3
16 c3 : titre = "Anti-Patrons"
17 c3 : pages = {"...", "..."}
18
19 object "_ : Chapitre" as c4
20 c4 : titre = "Conclusion"
21 c4 : pages = {"..."}
22
23 mA → "cs" c1
24 mA → "cs" c2
25 mA → "cs" c3
26 mA → "cs" c4
27
28 @enduml
```



MODÉLISATION : UML



```
1 @!include ../commons.style
2 @startuml dseq_example
3 participant "mémoireAlice: Mémoire" as mA
4 participant "ci: Chapitre" as ci
5 [-> mA: taille()
6 activate mA
7     mA -> mA : t = 0
8     loop ci in cs
9         mA -> ci: taille()
10        activate ci
11        mA ← ci: ti
12        deactivate ci
13        mA -> mA : t = t + ti
14    end
15 [-> mA: t
16 deactivate mA
17 @enduml
```



RAPPELS

- **interface** = **contrat**
un type abstrait
- **classe** = une **réalisation** (implémentation) possible **de contrat(s)**
réalise un ou plusieurs types abstraits
interface publique vs le reste
- **objet** = une **instance** de classe
une classe vue comme un « moule à objets »
accès à un service via appel de méthode

CAPTURE DU CORPUS DE L'EXPÉRIENCE

- **patrons de conception**, (très) nombreux
on cite souvent le « **Gang of Four** », GoF (Gamma et al, 1994)
proposition de 23 patrons regroupés en 3 grandes catégories
- patrons structurés en **4 parties** :

nom	(= vocabulaire commun)
problème	(= « quand ? »)
solution	(= « comment ? », éléments de conception abstraits)
conséquences	(= bénéfices, compromis à faire)
- même idée pour les défauts de conception : **anti-patrons** (aka odeurs de code)

SYNTHÈSE

- processus et solution de **conception**
- rarement « **la** » bonne solution
surtout si n dimensions de qualité
résultat de **décisions de conception**
- développement logiciel >> conception
intérêt de **bonnes pratiques**
dont DRY, KISS, SOLID, STUPID
- **capture** de connaissances
liées à un cas d'étude (dont **modèles** UML)
ou général (**patrons** et **anti-patrons**)

